

Problem Set 2: Divide & Conquer and Arithmetic

Devin Fromond

Due: September 6th 2024

- Please type your solutions using $\text{\LaTeX}$ or any other software. Handwritten solutions will not be accepted.
- Your algorithms must be in plain English & mathematical expressions, and the pseudo-code is optional. Pseudo-code, without sufficient explanation, will receive no credit.
- Unless otherwise stated, all logarithms are to base two.
- If we ask for a specific running time, a correct solution achieving it will receive full credit even if a faster solution exists.

Problem 1: An Upward Trajectory

(25 points)

You have been working hard to improve your grades recently, and are curious to determine whether you are succeeding or not. So you decide to take a list of your assignment grades in chronological order, and determine if they are showing an upward trajectory.

You want to find the number of times when, on any assignment, you exceeded your grade on a previous assignment. For example, if your grades were [82, 100, 70, 98, 100], you would output 6. Specifically, when you received a 100% on the 2nd assignment (1-indexed), that exceeded the 82% you received on the 1st assignment; when you received a 98% on the 4th assignment, that also exceeded the 82% you received on the 1st assignment plus the 70% on the 3rd assignment; and when you received a 100% on the last assignment, that exceeded your grades on the 1st, 3rd, and 4th assignment, creating a total of 6 instances of performance improvements.

Your goal is to design a divide-and-conquer algorithm for this problem that runs in time $\mathcal{O}(n \log n)$. Remember to justify your algorithm's correctness, show your recurrence, and prove runtime using the Master Theorem.

Hint: Is there a step in the traditional merge sort algorithm where we might be able to easily get this information?

Solution:

- *Algorithm:* We recognize counting the number of times where the grade exceeded the previous assignment's grade is synonymous to counting the number of inversions in the Merge Sort algorithm.

First, the algorithm will divide the array into two halves, a left half and a right half using the middle index, until each half only has one element.

While merging the two halves back in sorted order, the algorithm counts how many times elements from the right half exceeds the elements in the left half. This count represents the number of inversions required to sort the array.

Finally, the sum of the number of inversions are calculated which yields the answer.

- *Correctness:* Using similar logic from the Merge Sort correctness justification from lecture, we will assume arrays x and y are sorted; thereby the smallest element is the first element of the sorted array combining x and y . We will then assume $x[1] < y[1]$, establishing $x[1]$ is smaller than the smallest element of y . Therefore, when merging x and y , $x[1]$ is the first element in the sorted array.

This logic is applicable to an array of any length since the logic is, in essence, a base case for an array of size 1. Merge Sort divides the array in half until this base case is

reached, thereby proving its correctness.

One aspect we must prove, however, is that counting inversions are synonymous to counting the number of times where the grade exceeded the previous assignment's grade. As such, we will use the array [1, 2, 3, 4] to generalize. We can see the array is already sorted. Yet, merge sort will still calculate the number of times the right element is greater than the left element. As such, we can see 4 is greater than 3, 2, and 1. Similarly, 3 is greater than 2 and 1. Finally, 2 is greater than 1. In the following pseudo code from lecture:

```
def merge(x[1...k], y[1...m]):
    if x is length 0
        return y
    if y is length 0
        return x
    if x[1] <= y[1]:
        return x[1].join(merge(x[2...k], y[1...k]))
    else:
        inversions++
        return y[1].join(merge(x[1...k], y[2...k])
```

As seen, the correctness of inversions is proven since, without loss of generality, the sample array logic can be broken down to smaller arrays and applied.

- *Time Complexity Analysis:* Relatively trivially, we can see each recursive level performs $O(n)$ work during the merge step. Furthermore, we find there are two recursive problems and the size of the sub-problem is $n/2$. As such, we have a general form of $T(n) = 2T(n/2) + O(n)$. Therefore, we easily find - by the Master's Theorem - the time complexity is $O(n \log(n))$ since $\log_2(2) = 1$.

Problem 2: Emptying Tanks

(25 points)

Aqua has been assigned the task of emptying n water tanks before the supervisors return in h hours. The i -th tank contains $tanks[i]$ liters of water, where $tanks$ is an n -sized array.

She can choose how fast to drain the water, with a fixed rate of r liters per hour. Each hour, Aqua picks one tank and drains up to r liters from it. If the tank has less than r liters left, she drains the remaining water and does not drain any more that hour.

Aqua prefers to take her time but must ensure that all the tanks are empty before the supervisors return.

Using a divide and conquer approach, find the minimum rate r that Aqua needs to set in order to empty all the tanks within the given h hours. Justify your algorithm's correctness and runtime.

Solution:

- *Algorithm:* We know the rate r must be an integer (from EdStem). Therefore, we can establish two variables according to some loose rules:

$r_{min} = 1$, since 1 is the minimum integer the flow rate can be

$r_{max} = \max(tanks)$, since the maximum flow rate needed would be to drain the largest tank in one hour

Therefore, we define the method `isEmpty(tanks, r, h)` to calculate if it is possible to empty all the tanks using the flow rate r in h hours. This method will define a *totalHours* variable then will iterate through each index of the *tanks* array and calculate the total hours to drain the tank, adding the hours necessary to *totalHours*. Then, the method will return **True** if $totalHours \leq h$ or **False** otherwise.

The other method will determine the minimum drain rate by defining the r_{min} and r_{max} as previously defined then keep finding the midpoint of r_{min} and r_{max} while checking if this midpoint is valid given the *isEmpty* method. If it is not valid, the method will find the midpoint of the midpoint and r_{max} . If the midpoint is valid, then the method will find the midpoint of r_{min} and midpoint. To clarify, a midpoint is valid based off of the result of *isEmpty*. Ultimately, this method will divide the range of r in half each time to find the minimum flow rate r that will satisfy the problem's requirements.

- *Correctness:* The binary search approach of identifying a midpoint and searching for the next midpoint based off of the validity of the midpoint ensures the optimal rate of r is explored.

Abstractly, this is simply having an array of size $\max(tanks)$ where the elements, starting with 1, increment by 1 then doing binary search on this array with a verification method.

As a base case, we have the difference between a midpoint and a the checked bound being 0. Therefore, we have found the minimum value of r such that *isEmpty* is **True**. We know this base case will be arrived to since the binary search halves the search area each iteration.

- *Time Complexity Analysis:* At each level, we must iterate through the elements in *tanks* to determine the validity of the *isEmpty* function. Therefore, the work done at each level is $O(n)$. The number of levels is directly influenced by the size of the largest tank. Using binary search, the time complexity to search for one value is expressed as $O(\log(\max(\text{tanks})))$. As such, since the Master's Theorem provides a loose definition of time complexity to be the work done at each level * the number of levels, we arrive at our answer of $O(n \log(\max(\text{tanks})))$.

While this answer does not follow the traditional path of the Master's Theorem, it is nonetheless a divide and conquer algorithm since - according to EdStem - "iterative binary search would still be divide and conquer, because even though it is not recursive, it still begins with a problem of input size N that solves a single sub-problem of input size $N/2$ and then merges sub-problem solutions... and returns the final answer". It should be noted that, depending on the level of abstraction we wish to use, our time complexity may be highly abstracted to $O(n \log(n))$.

Problem 3: Maximum Bananas

(25 points)

A monkey currently has b rotten bananas and seeks to get rid of as much bananas as possible. There are n portals laid out in order and each portal has a multiplier that can be applied on the current amount of bananas the monkey has. For example, going through a portal with a multiplier of 0.5 will halve the amount of rotten bananas the monkey has. These multipliers are represented as a 1-indexed array of non negative numbers A of length n . The monkey is allowed to start at some portal $1 \leq i \leq n$, applying multipliers to his bananas up to some index $j \geq i$ *without* skipping indices. For example, given the array $A = [3, 0.1, 0.2, 100, 0.75, 4, 0.1]$ and $b = 100$ bananas, the monkey's best course of action would be to start at position 2 and stop at position 3. The maximum number of bananas that the monkey could get rid of is $100 - (100 \cdot 0.1 \cdot 0.2) = 98$.

Design an $O(n \log n)$ Divide & Conquer algorithm to find the maximum amount of bananas the monkey can get rid of. Justify correctness of your algorithm, provide a recurrence, and prove its runtime. **You can assume that multiplication is fixed point and is $O(1)$.**

Solution:

- *Algorithm:* Through simple algebraic manipulation, we find the algorithm must maximize y , where y is defined as $100 - (100 * elements) = 100 - 100(elements) = 100(1 - elements)$. Therefore, it becomes apparent to maximize y , we must minimize *elements*. As such, the algorithm we wish to produce searches for the smallest sub-array of A .

We will utilize an iterative approach of binary search to complete the algorithm. We will define a method `smallestProduct(len, array)` that computes the minimum product of a sub-array of len elements for a given array. This method will utilize a sliding window approach that uses a window of size len to determine minimum product. The sliding window calculates the minimum product by calculating the product of the first len elements, storing the value as a `minProduct` if necessary, increasing the window index, then repeating until the end of the array is reached.

Initially, the other method will calculate the midpoint of the length of the array then call the *smallestProduct* method. If the `minProduct` of this length yields a smaller product than our previously defined `minProduct`, shorter lengths are checked; the midpoint of the first half of the array is calculated and used in the *smallestProduct* method. Otherwise, we check larger lengths since we have not found the smallest product.

This process ends until the left and right pointers converge, meaning the smallest sub-array yielding the smallest product has been found.

- *Correctness:* The algorithm correctly finds the smallest sub-array by searching all possible (reasonable) sub-array sizes. It searches all possible sub-array lengths from 1 to

n , halving the search space in each step.

Regarding the sliding window, this ensures all continuous sub-arrays are considered and the minProduct is successfully calculated - no matter the size of the array.

By combining a binary search methodology to the length search and a sliding window, we ensure all potential and possible smallest sub-arrays are calculated and compared.

- *Time Complexity Analysis:* As mentioned previously, the binary search component of the algorithm takes an input of size n and divides the work into 2 subsections of size $n/2$. Additionally, by the sliding window, the work done at each level is $O(n)$. By the Master's Theorem, we find $a = 2, b = 2, d = 1$, and so, $T(n) + 2T(n/2) + O(n)$ yields a time complexity of $O(n\log(n))$ as desired.

Problem 4: Laptop Sleeves

(25 points)

You have n laptops of different sizes and n differently sized laptop sleeves. There is one-to-one relationship between each laptop and their corresponding, perfectly-fitting laptop sleeve. You can try to match each laptop to the right sleeve by determining whether the laptop is larger than the sleeve, is smaller than the sleeve, or fits exactly in the sleeve. **However, there is no way to compare two laptops or two sleeves together.** The problem is to match each laptop to its sleeve. Design a randomized divide-and-conquer algorithm to solve this problem with an expected $O(n \log n)$ running time. Give a brief explanation of your algorithm's correctness and runtime.

Solution:

- *Algorithm:* We will proceed by using a semi-sorted array using a random laptop sleeve s_{ref} . Using the reference sleeve, we will compare each laptop to the sleeve and place it in the array accordingly. If $laptop < s_{ref}$, it will go in the array *smallerArray*. If $laptop > s_{ref}$, it will go in the array *largerArray*. This process is $O(n)$ each time we select s_{ref} and sort the array. If $laptop = s_{ref}$, we will store the pair in a new array called *returnArray*.

The next sleeve will be chosen at random and will be called s_{ref} . The sleeve will be compared to the laptop in the *returnArray*. If $laptop > s_{ref}$, then s_{ref} will be compared to the *smallerArray*. The aforementioned process of sorting will happen. The reverse is true if $laptop < s_{ref}$. This process will, on average, take $O(\log(n))$ time to run since the algorithm divides the algorithm into two arrays of the same average size.

The aforementioned process repeats recursively. Here's a useful, less step-by-step, algorithm description:

In a `classifyLaptop(laptops, sleeves, s_ref)` method, the laptops are iterated through, compared to s_{ref} , and put into the respective array. This method returns all three arrays.

In a different method, *sleeves* is iterated through as to choose a new reference each time. A call to `classifyLaptops` is made, thereby updating the three arrays. There are two recursive calls: one for the smaller array and one for the larger array. These recursive calls further divide and sort the two arrays (*smallerArray* and *largerArray*), breaking the array into smaller and smaller sub-problems. After the recursive calls are done, we remove the reference from the array. Finally, the method ends and returns the pairs if $len(returnedArray) == len(laptops)$.

This algorithm is basically a modified version of Quicksort where the pivot is replaced with the reference sleeve.

- *Correctness:* The base case of the algorithm is correctly classifying the laptop into smaller, larger, or the same size as the sleeve when there is one sleeve. Since the algorithm is recursive where the arrays get increasingly smaller, we know we must arrive

at this base case. As such, we will assume it to be true.

We know through previous explanation laptops classified as smaller than the reference sleeve are compared to and matched with sleeves smaller than the reference sleeve, so this ensures each laptop gets matched with the correct sleeve. Similarly, the opposite is true.

After the recursive calls, the results are combined with the exact matches. Since the exact matches and recursive calls are correct, the final result will be correct.

Therefore, we see by induction the algorithm correctly matches n laptops to its corresponding sleeve.

- *Time Complexity Analysis:* As previously established, we know the work completed at each level is $O(n)$ since we must iterate through each sleeve and classify each laptop into one of three arrays based on its relative size to the reference sleeve.

In each recursive step, the algorithm partitions the laptops into two sub-arrays, thereby creating two sub-problems whose size is approximately half of the original problem, assuming an average partition of $1/2$. Therefore, by the Master's Theorem, we can see $a = 2$ and $b = 2$. As such, we have $T(n) = 2T(n/2) + O(n)$ which leads us to our final time complexity of $O(n\log(n))$.